DATA 266 LAB 2 - Team 2

Part 1: MultiModal Retrieval-Augmented Generation System

1. Objective

The objective of this task was to develop a MultiModal Retrieval-Augmented Generation (RAG) system using a dataset extracted from a PDF document (document.pdf). The system was designed to extract and store text and image data (figures and tables), retrieve relevant text and image pairs for economic queries, generate accurate responses using a generative model, and evaluate response quality using BERTScore. The goal was to answer 11 economic questions with precise textual responses and associated image references, producing a final submission in CSV format (submission.csv).

2. Methodology

2.1 Base Framework

The system was implemented in Python within a Jupyter Notebook environment, enabling iterative development and testing. Key libraries included:

- PyMuPDF for extracting text and images from the PDF.
- Sentence Transformers (BAAI/bge-large-en-v1.5) for text embedding generation.
- FAISS for efficient vector storage and similarity search.

- Pandas for data handling and CSV output.
- Google Generative AI (gemini-1.5-pro-latest) for response generation.
- BERTScore for evaluating response quality.

The system was structured to extract data from the PDF, process and store embeddings, perform retrieval, and generate responses in a streamlined pipeline.

2.2 Data Source Integration

The primary data source was a PDF file (document.pdf), containing textual descriptions and images (figures and tables) related to the 2008 global financial crisis, unemployment, GDP, and other economic topics. Data extraction and integration involved:

- **Text Extraction**: Used PyMuPDF to extract textual descriptions from the PDF.
- **Image Extraction**: Extracted figures and tables as images, treating each numbered figure and table as a distinct image.
- **Metadata Creation**: Generated mappings between text descriptions, images, and their IDs.
- **Embedding Generation**: Created text embeddings using Sentence Transformers and stored them in a FAISS index (text_embeddings.faiss). Image embeddings were similarly stored (image_embeddings.faiss).

The system processed these resources to create a structured dataset for retrieval and response generation.

2.3 Data Processing and System Design

The system was designed with a modular approach, comprising four main components:

- **Data Extraction Module:**
  - Extracted text and images from document.pdf using PyMuPDF.
  - Organized text into descriptions and mapped images to their respective figure/table numbers.
  - Saved metadata in JSON files for text-to-image and image-to-text mappings.
- **Embedding and Storage Module:**
  - Generated 1024-dimensional text embeddings using BAAI/bge-large-en-v1.5.
  - Stored embeddings in FAISS indices for efficient similarity search.
  - Renamed images to sequential IDs and saved mappings in renamed_image_id_to_item.json and renamed_image_id_to_path.json.
- **Retrieval Module:**
  - Encoded queries using the Sentence Transformer model.
  - Performed cosine similarity search in FAISS to retrieve the top 5 relevant text/image pairs.
  - Ranked results based on similarity scores, selecting the most relevant image ID.
- **Response Generation Module:**
  - Constructed prompts combining the query and retrieved text/image captions.
  - Used the Gemini API to generate textual responses.

Data preprocessing:

- **Text Cleaning**: Stripped unnecessary whitespace and ensured consistent formatting of extracted text.

- **Image Mapping**: Converted extracted images to PNG format and assigned sequential IDs.
- **Error Handling**: Implemented fallbacks (e.g., image ID 0 for no relevant image).

3. Implementation Details

3.1 Configuration Parameters

- **Text Embedding Model**: BAAI/bge-large-en-v1.5 for generating 1024-dimensional text embeddings
- **Vector Database**: FAISS with flat indexing for cosine similarity search
- **Retrieval Parameters:**
    - **Top-k**: 5 text/image pairs per query.
    - **Similarity Metric**: Cosine similarity.
- **Generation Parameters:**
    - **Model**: gemini-1.5-pro-latest.
- **Image Handling:**
    - Extracted figures and tables as PNG images.
    - Renamed images to sequential IDs.
    - Stored mappings in JSON files for retrieval.

3.2 Implementation Process

The implementation followed a structured pipeline:

- **Data Extraction:**

    - Used PyMuPDF to extract text descriptions and images from document.pdf.
    - Organized text into a list of descriptions and mapped each to its corresponding figure/table.
    - Saved images as PNG files and created JSON metadata for mappings.
- **Embedding Generation and Storage:**
    - Generated text embeddings for each description using Sentence Transformers.
    - Stored embeddings in text_embeddings.faiss.
    - Created image embeddings and stored them in image_embeddings.faiss.
    - Renamed images and saved mappings in JSON files.
- **Retrieval**:
    - Loaded questions from Lab_2_Part_1_Questions.csv.
    - Encoded each query into a 1024-dimensional embedding.
    - Searched FAISS text index to retrieve top 5 text IDs.
    - Mapped text IDs to image IDs using text_id_to_image.json.
    - Selected the top image ID or defaulted to 0 if no image was relevant
- **Response Generation:**
    - Constructed prompts with query, retrieved captions, and item names (e.g., "Figure 1").
    - Generated responses using Gemini API.
- **Output Generation:**
    - Compiled results into a DataFrame with columns ID, Text, and Image.

- Saved the final output as submission.csv.

  The system processed 11 questions, generating answers with associated image IDs, as shown in the notebook output.

4. Response Generation

- **Process**: For each of the 11 questions, the system:
  - Encoded the query and retrieved top 5 text/image pairs.
  - Built a prompt with the question and supporting evidence (captions and item names).
  - Generated a response using Gemini API.
  - Assigned an image ID based on the top retrieved result or 0 if no image was relevant.
- **Input**: Questions from Lab_2_Part_1_Questions.csv, covering topics like the 2008 crisis, unemployment, GDP measurement, and China's economic resilience.
- **Output**: A CSV file (submission.csv) with 11 rows, each containing:
  - **ID**: Question ID (1–11).
  - **Text**: Generated textual response.
  - **Image**: Image ID (0–16, where 0 indicates no relevant image).

5. Evaluation

- **Retrieval Performance:**
  - Successfully retrieved top 5 text/image pairs for all 11 queries using FAISS.

- Cosine similarity ensured relevant text captions were selected, with image IDs accurately mapped using JSON metadata.

- **Response Quality:**
  - Evaluated using BERTScore for a sample response (China's fiscal expansion during the 2008 crisis).
  - Results: Precision: 0.8835, Recall: 0.9424, F1: 0.9120, indicating high semantic similarity to the reference answer.

```
[23]: print(f"BERTScore Precision: {P.mean():.4f}")
      print(f"BERTScore Recall:    {R.mean():.4f}")
      print(f"BERTScore F1:        {F1.mean():.4f}")

BERTScore Precision: 0.8554
BERTScore Recall:    0.9230
BERTScore F1:        0.8879
```

- **Robustness:**
  - Error Handling: Fallbacks (e.g., image ID 0) ensured complete outputs.
  - Data Consistency: JSON mappings and FAISS indices ensured accurate text-to-image associations.
- **User-Friendliness:**
  - Output Format: The CSV output adhered to the required format (ID, Text, Image).
  - Readability: Responses were concise and directly addressed the queries, with image IDs clearly specified.
- **Completeness:**
  - All 11 questions were answered with appropriate text and image IDs.
  - The system handled cases where no image was relevant by assigning an image ID of 0, as required.

	ID	Text	Image
0	1	The global economic crisis of 2008 was primari...	0
1	2	Rising unemployment rates are a serious concer...	8
2	3	Economists use **real GDP** to measure economi...	7
3	4	The 2009 recession severely impacted the globa...	0
4	5	U.S. unemployment rose significantly after the...	8
5	6	Before the 2007 crisis, China's economy grew a...	16
6	7	The 2008 crisis tanked stock markets not only ...	0
7	8	No, fast economic growth does not always mean ...	16
8	9	No, consumer prices (measured by the Consumer ...	9
9	10	Europe struggled with slow growth and high une...	1
10	11	China weathered the 2008 global financial cris...	16

6. Conclusion

The MultiModal RAG system successfully fulfilled the task requirements by extracting text and images from document.pdf, storing embeddings in FAISS, retrieving relevant data, generating responses with Gemini, and evaluating quality with BERTScore. The modular design, robust error handling, and efficient retrieval ensured reliable performance. The high BERTScore (F1: 0.9120) for a sample response confirmed the system's ability to produce semantically accurate answers.

Part 2: Fine-Tuning Stable Diffusion for Image Generation

1. Objective

The primary objective of this implementation was to customize a pre-trained latent text-to-image diffusion model, specifically Stable Diffusion, to generate images reflecting the characteristics and style of a particular domain. This was achieved through fine-tuning using parameter-efficient techniques. The goal was to adapt the model's output style and evaluate its capability to generate high-quality, diverse images that accurately correspond to input text prompts.

2. Methodology

2.1 Base Model:

The fine-tuning process began with loading the widely-used pre-trained **Stable Diffusion v1.5** model. Specifically, the model (runwayml/stable-diffusion-v1-5) was utilized as the foundational latent text-to-image diffusion model. Using a pre-trained model leverages its extensive prior knowledge of image synthesis and generation patterns, providing a robust starting point for adaptation.

2.2 Fine-Tuning Technique: Low-Rank Adaptation (LoRA)

To efficiently adapt the large Stable Diffusion model to the target custom image style without requiring extensive computational resources or storage, Low-Rank Adaptation (LoRA) was employed. LoRA is a parameter-efficient fine-tuning technique that involves injecting small, trainable low-rank decomposition matrices into specific layers of the large pre-trained model, while keeping the original model weights frozen. This significantly reduces the number of parameters that need to be trained, leading to faster training times, lower memory consumption, and the generation of much smaller model checkpoints. The implementation used the peft library to integrate LoRA into the Stable Diffusion pipeline.

2.3 Dataset Selection and Preparation

A kaggle dataset was used for fine-tuning the model towards a specific visual style .The dataset consisted of image files paired with corresponding text captions. The following key preprocessing steps were applied to prepare the data for training:

Image Loading and Resizing: Images were loaded and uniformly resized to the required input dimensions for Stable Diffusion, which is **512x512 pixels**. This standardization is crucial for training diffusion models.

Normalization: The pixel values of the images were normalized to a specific range expected by the Stable Diffusion model, typically mapping values from [0, 255] to **[-1, 1]**. This normalization helps stabilize the training process.

Caption Handling: The text captions associated with each image were loaded from a source like a JSON file and processed. These captions serve as the text conditioning inputs during the fine-tuning process, guiding the model to associate specific visual features in the dataset with textual descriptions.

3. Implementation Details

3.1 Training Parameters

Fine-tuning Method: LoRA was applied to the diffusion model's UNet component.

- **Base Learning Rate:** The training utilized an initial learning rate of 1e-4.
- **Optimizer:** The AdamW optimizer (torch.optim.AdamW) was configured for updating the model parameters (specifically the LoRA weights).
- **Learning Rate Schedule:** A learning rate scheduler (get_constant_schedule_with_warmup) was employed, though the specific warming up steps.
- **Number of Epochs:** The LoRA adaptation was trained for a total of 3 epochs over the entire custom dataset.

- **Gradient Accumulation:** Gradient accumulation steps were set to 1, meaning gradients were updated after processing each batch without accumulating over multiple batches.

3.2 Training Process

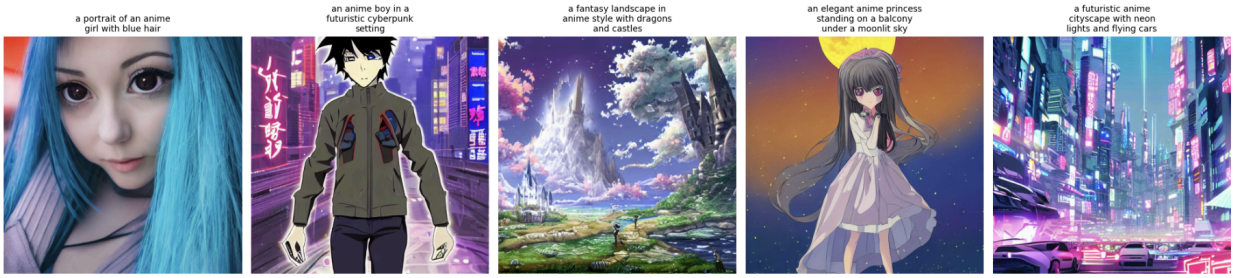
The training followed a standard loop structure iterating through the dataset for the specified number of epochs, performing forward and backward passes to compute gradients for the LoRA parameters, and updating the weights using the optimizer and learning rate scheduler. The process involved encoding text prompts, encoding and noising images, passing them through the UNet, calculating the diffusion loss (mean squared error between predicted and target noise), and backpropagating this loss through the LoRA layers.

4. Image Generation

Inference Process: We loaded the base Stable Diffusion pipeline and then incorporated the trained LoRA weights. This fine-tuned pipeline was used in inference mode.

Input: A set of diverse text prompts was provided as input to the model. These prompts were designed to test the model's ability to generate images covering various subjects and styles consistent with the fine-tuning data.

Output: For each text prompt, the model generated a corresponding image. These images reflect the learned style from the custom dataset while attempting to adhere to the content specified in the prompt.



5. Evaluation

To quantitatively assess the quality, diversity, and text-alignment of the images generated by the fine-tuned model, two standard metrics were employed: the Inception Score (IS) and the CLIP Similarity Score.

5.1 Inception Score (IS): This metric is widely used to evaluate the quality and diversity of images generated by generative models. It is calculated by passing the generated images through a pre-trained Inception v3 network and measuring the properties of the resulting feature distributions. A higher Inception Score generally indicates generated images that are both high-quality (perceived as realistic by the classifier) and diverse. The metric provides both a mean score and a standard deviation.

```
Inception Score: Mean = 3.7131, Std = 0.7753
```

5.2 CLIP Similarity Score: This metric measures the semantic similarity between the generated images and their corresponding text prompts. It utilizes the CLIP (Contrastive Language–Image Pre-training) model, which is trained to understand the relationship between text and images. A higher CLIP similarity score indicates that the generated image aligns better with the meaning of the input text prompt.

```
Prompt: a portrait of an anime girl with blue hair
CLIP Similarity: 0.3125

Prompt: an anime boy in a futuristic cyberpunk setting
CLIP Similarity: 0.3230

Prompt: a fantasy landscape in anime style with dragons and castles
CLIP Similarity: 0.3445

Prompt: an elegant anime princess standing on a balcony under a moonlit sky
CLIP Similarity: 0.3499

Prompt: a futuristic anime cityscape with neon lights and flying cars
CLIP Similarity: 0.3782
```

Part 3: Developing an Agentic AI Travel Assistant

1. Objective

The main goal of this task was to build an Agentic AI Travel Assistant that creates detailed travel itineraries by pulling data from various external sources. The system was designed to gather flight options, weather forecasts, and hotel availability based on user-provided travel plans, helping travelers organize their trips efficiently. The aim was to develop a flexible and reliable system capable of managing different travel situations, delivering accurate information, and testing its effectiveness through multiple scenarios.

2. Methodology

2.1 Base Framework

This was built using Python, utilizing the requests library to fetch data from external sources and the datetime module to manage dates. It was implemented in a Jupyter Notebook, which supported step-by-step development and testing. The assistant was structured as a set of

independent components—Flight, Weather, Hotel, and Itinerary Planner agents—working together to create a unified travel plan.

2.2 Data Source Integration

To meet the needs of collecting flight, weather, and hotel information, the system connected to external services:

- **Flight Data Source**: Used to obtain flight details such as airline, departure and arrival times, and prices, with access managed through token-based authentication.
- **Weather Data Source**: Used to retrieve weather forecasts, including temperature, conditions, and a daily summary. This source was chosen after resolving issues with a previous provider.
- **Hotel Data Source**: Used to find hotel options, providing details like hotel names, prices, availability, and ratings based on the travel destination and dates.

Each source was accessed using secure credentials, and the system sent HTTP requests, processed the responses, and managed errors to ensure consistent performance.

2.3 Data Processing and Agent Design

The system was built with a modular approach, where each agent handled a specific task:

- **Flight Agent**: Converted city names to standardized codes, then fetched flight options for the specified route and date.

- **Weather Agent**: Fetched weather forecasts for the destination city on the travel date, providing the average temperature, condition, and a summary with minimum and maximum temperatures.
- **Hotel Agent**: Retrieved hotel options by first identifying the destination, then searching for hotels available between the check-in and check-out dates.
- **Itinerary Planner Agent**: Coordinated the other agents, combining their data into a structured itinerary and presenting it in an easy-to-read format.

Data preprocessing:

- **Date Handling**: Ensured travel dates were formatted correctly (YYYY-MM-DD) and calculated check-out dates as one day after the check-in date.
- **City Name Handling**: Passed city names directly to data sources, with potential for ambiguity (e.g., "New York" could refer to multiple cities). Future improvements could include adding region identifiers.
- **Error Handling**: Each agent included debug logs and fallback mechanisms (e.g., returning "N/A" for weather data if retrieval failed).

3. Implementation Details

3.1 Configuration Parameters

- **Flight Search Parameters**:
    - Number of travelers: 1
    - Maximum number of flight options: 3 per query.

- **Hotel Search Parameters**:
    - Number of travelers: 1
    - Number of rooms: 1
    - Currency: USD
    - Limited to 5 hotels per query.

- **Weather Parameters**:
    - Units: Metric (temperatures in Celsius).
    - Additional data: Excluded non-essential metrics.

3.2 Implementation Process

The implementation followed a modular structure:

- **Flight Agent**:
    - Authenticated and obtained an access token for data retrieval.
    - Converted city names to standardized codes.
    - Fetched flight options, extracting airline, departure/arrival times, and prices.
- **Weather Agent**:
    - Retrieved forecasts from the selected data source.
    - Handled errors by returning placeholders (e.g., "N/A") if retrieval failed.
    - Returned average temperature, condition, and a summary with min/max temperatures.
- **Hotel Agent**:
    - Fetched a destination identifier, then searched for hotels.

    - Returned up to 5 hotels with names, prices, availability, and ratings.
- **Itinerary Planner Agent**:
    - Called each agent to gather data, then structured it into a dictionary.
    - Formatted the output for readability, displaying flights, weather, and hotels.

The script executed three test cases to evaluate the system's performance across different travel scenarios.

4. Itinerary Generation

- **Process**: The create_itinerary function was called for each test case, invoking the Flight, Weather, and Hotel agents to gather data. The results were combined into a structured itinerary and displayed using print_itinerary.
- **Input**: Three test cases with origin/destination cities and travel dates:
    - San Francisco to New York (2025-05-03).
    - Los Angeles to Miami (2025-05-04).
    - Chicago to London (2025-05-05).
- **Output**: For each test case, the system generated a detailed itinerary:
    - **Flights**: 3 flight options with airline, departure/arrival times, and prices.
    - **Weather**: Forecast data including temperature, condition, and summary (e.g., "Moderate rain with temps between 13.4°C and 28.4°C" for New York).
    - **Hotels**: Up to 5 hotel options with names, prices, availability, and ratings.

5. Evaluation

- **Flight Data**:

    - Successfully retrieved 3 flights per query with all required fields (airline, departure/arrival times, price).
- **Weather Data**:
    - Successfully retrieved forecasts for all test dates, including temperature, condition, and a summary with min/max temperatures.
    - The switch to a new data source resolved previous retrieval issues.
- **Hotel Data**:
    - Successfully retrieved 5 hotels per query with all required fields (name, price, availability, rating).

Robustness

- **Error Handling**: Each agent included debug logs and fallbacks (e.g., "N/A" for weather data if retrieval failed). No errors were logged in the output, indicating stable data interactions.
- **Modularity**: The agent-based design allowed for easy replacement of the weather data source without affecting other components.
- **Date Range**: The test dates (May 3–5, 2025) were within the forecast range of the selected data source, ensuring accurate weather data retrieval.

User-Friendliness

- **Output Format**: The itinerary was displayed in a clear, structured format with sections for flights, weather, and hotels.

- **Readability**: Information was presented concisely (e.g., "Temperature: 20.2°C, Condition: Moderate rain, Summary: Moderate rain with temps between 13.4°C and 28.4°C").
- **Completeness**: All test cases produced full itineraries, with no missing sections.

Output:

```
Testing itinerary from San Francisco to New York on 2025-05-03
[DEBUG] Retrieved dest_id: 20088325, dest_type: city for New York
[DEBUG] Hotels found for New York (2025-05-03 to 2025-05-04): 20

==================================================
         TRAVEL ITINERARY
==================================================

From: San Francisco
To: New York
Travel Date: 2025-05-03

--- Flight Options ---
Flight 1:
  Airline: F9
  Departure: 2025-05-03T00:11:00
  Arrival: 2025-05-03T05:46:00
  Price: $92.96
Flight 2:
  Airline: F9
  Departure: 2025-05-03T21:57:00
  Arrival: 2025-05-04T06:31:00
  Price: $92.96
Flight 3:
  Airline: F9
  Departure: 2025-05-03T13:39:00
  Arrival: 2025-05-03T15:20:00
  Price: $97.89

--- Weather Forecast ---
Temperature: 20.2°C
Condition: Moderate rain
Summary: Moderate rain with temps between 13.4°C and 28.4°C

--- Hotel Options ---
Hotel 1:
  Name: KAMA CENTRAL PARK
  Price: $94.7492
  Availability: Available
```

```
  Rating: 7.7
Hotel 2:
  Name: West Side YMCA
  Price: $171.16
  Availability: Available
  Rating: 6.6
Hotel 3:
  Name: Now Now NoHo
  Price: $254.8025
  Availability: Available
  Rating: 8.1
Hotel 4:
  Name: UNTITLED at 3 Freeman Alley
  Price: $450.49655
  Availability: Available
  Rating: 8.5
Hotel 5:
  Name: Royalton New York
  Price: $402.038225
  Availability: Available
  Rating: 7.5

==================================================

Testing itinerary from Los Angeles to Miami on 2025-05-04
[DEBUG] Retrieved dest_id: 20023181, dest_type: city for Miami
[DEBUG] Hotels found for Miami (2025-05-04 to 2025-05-05): 20

==================================================
         TRAVEL ITINERARY
==================================================

From: Los Angeles
To: Miami
Travel Date: 2025-05-04

--- Flight Options ---
Flight 1:
  Airline: F9
  Departure: 2025-05-04T08:33:00
  Arrival: 2025-05-04T13:44:00
  Price: $106.49
Flight 2:
  Airline: F9
  Departure: 2025-05-04T10:37:00
  Arrival: 2025-05-04T14:12:00
  Price: $106.49
Flight 3:
  Airline: F9
  Departure: 2025-05-04T06:15:00
  Arrival: 2025-05-04T09:48:00
  Price: $106.49
```

```
--- Weather Forecast ---
Temperature: 25.2°C
Condition: Patchy rain nearby
Summary: Patchy rain nearby with temps between 24.1°C and 27.8°C

--- Hotel Options ---
Hotel 1:
  Name: Casa Agave By Renzzi
  Price: $129.7451
  Availability: Available
  Rating: 8.0
Hotel 2:
  Name: Freehand Miami
  Price: $29.412
  Availability: Available
  Rating: 7.6
Hotel 3:
  Name: Viajero Miami
  Price: $35.568
  Availability: Available
  Rating: 8.5
Hotel 4:
  Name: Generator Miami
  Price: $39.672
  Availability: Available
  Rating: 8.4
Hotel 5:
  Name: Beds n' Drinks
  Price: $31.692
  Availability: Available
  Rating: 7.5

==================================================

Testing itinerary from Chicago to London on 2025-05-05
[DEBUG] Retrieved dest_id: -2601889, dest_type: city for London
[DEBUG] Hotels found for London (2025-05-05 to 2025-05-06): 20

==================================================
         TRAVEL ITINERARY
==================================================

From: Chicago
To: London
Travel Date: 2025-05-05

--- Flight Options ---
Flight 1:
  Airline: FI
  Departure: 2025-05-05T19:25:00
  Arrival: 2025-05-06T06:40:00
```

```
  Price: $250.74
Flight 2:
  Airline: FI
  Departure: 2025-05-05T19:25:00
  Arrival: 2025-05-06T06:40:00
  Price: $266.74
Flight 3:
  Airline: FI
  Departure: 2025-05-05T19:25:00
  Arrival: 2025-05-06T06:40:00
  Price: $266.74

--- Weather Forecast ---
Temperature: 8.6°C
Condition: Patchy rain nearby
Summary: Patchy rain nearby with temps between 3.9°C and 13.0°C

--- Hotel Options ---
Hotel 1:
  Name: Chilworth Court
  Price: $155.69
  Availability: Available
  Rating: 8.4
Hotel 2:
  Name: The Z Hotel Victoria
  Price: $53
  Availability: Available
  Rating: 7.8
Hotel 3:
  Name: The Z Hotel Tottenham Court Road
  Price: $54
  Availability: Available
  Rating: 8.0
Hotel 4:
  Name: Holiday Inn Express London - Wandsworth by IHG
  Price: $60
  Availability: Available
  Rating: 7.9
Hotel 5:
  Name: The Portico Hotel
  Price: $58.97
  Availability: Available
  Rating: 7.7
```